

IOT FISH POND MANAGEMENT SYSTEM

ENGINEERING IMPLEMENTATION GUIDE

*Complete Build Guide: Wiring, Firmware, App Development & System
Configuration*

For Internal Engineering Team Use Only
Document Version 1.0

1. Project Overview & Engineering Scope

This guide covers the full implementation of the IoT-Based Fish Pond Monitoring and Management System from the engineering perspective. It is organized into five major phases: hardware assembly and wiring, ESP32 firmware development, mobile application development, Firebase cloud backend configuration, and full system integration and testing.

The system uses an ESP32 as the central controller connected to three sensors (DS18B20 temperature, PH4502C pH, Turbidity), two relay-driven actuators (aerator and feeder motor), and a local alert circuit (buzzer + LED). All data flows through Firebase Realtime Database to a Flutter-based mobile application.

Layer	Technology
Device / Firmware	C++ (Arduino), MQTT (PubSubClient)
MQTT Broker	Mosquitto (lightweight, self-hosted)
Backend	Node.js + Express.js
Real-Time Sync	Socket.IO
Database	MongoDB (Atlas or self-hosted)
Frontend / App	React + Tailwind CSS (Progressive Web App)
Authentication	JWT + Firebase Auth
Push Notifications	Web Push API (VAPID keys)

2. System Architecture Overview

This engineering guide describes the complete implementation of the IoT Fish Pond Monitoring and Management System using the updated technology stack. Flutter has been replaced entirely. The new stack uses Node.js/Express.js as the backend, Mosquitto as the MQTT broker, MongoDB as the database, and a React-based Progressive Web App (PWA) as the user-facing application.

2.1 Architecture Diagram (Text Representation)





2.2 Communication Protocol

The ESP32 communicates exclusively via MQTT over Wi-Fi. MQTT is chosen because it is lightweight, supports publish/subscribe patterns, and is perfectly suited for IoT devices with intermittent connectivity.

Topic	Direction	Purpose
fishpond/{id}/sensors	ESP32 → Broker → Server	Publish sensor readings (JSON)
fishpond/{id}/commands	Server → Broker → ESP32	Send control commands
fishpond/{id}/status	ESP32 → Broker → Server	Device heartbeat / online status
fishpond/{id}/alerts	ESP32 → Broker → Server	Local threshold breach event
fishpond/{id}/config	Server → Broker → ESP32	Push updated config/schedules

3. Bill of Materials (BOM) with Exact Component Details

Procure all components before beginning assembly. Verify component specifications against this table.

Component	Qty	Price (BDT)	Supplier	Key Spec
ESP32 Dev Board (30-pin)	1	430	roboticsbd.com	Dual-core, built-in WiFi
DS18B20 Waterproof	1	220	daraz.com.bd	Digital, OneWire protocol

Component	Qty	Price (BDT)	Supplier	Key Spec
PH4502C pH Module	1	2,900	techshopbd.com	Analog out, 0-14 pH range
Turbidity Sensor Module	1	820	techshopbd.com	Analog out
2-Channel Relay Module	1	400	roboticsbd.com	5V coil, 10A 250VAC
Buzzer (Active)	1	80	Local electronics	5V active buzzer
LED (Red, 5mm)	1	10	Local electronics	5mm red LED
330-ohm Resistor	1	5	Local electronics	For LED current limiting
4.7k-ohm Resistor	1	5	Local electronics	DS18B20 pull-up
18650 Battery	1	80	roboticsbd.com	3.7V 2600mAh
TP4056 Charger Module	1	60	Local electronics	5V USB input
Aquarium Air Pump	1	~300	Local pet shop	220V AC small pump
DC Motor (for feeder)	1	~200	Local shop	5-12V DC geared motor
Breadboard (full size)	1	120	Local electronics	830 tie-points
Jumper Wires (M-M, M-F)	1 set	80	Local electronics	20cm assorted
USB Micro cable	1	80	Local shop	For ESP32 power/prog
pH buffer solution set	1 set	~200	Chemistry supplier	pH 4.0, pH 7.0
Waterproof project box	1	~150	Local hardware	For electronics enclosure

4. Hardware Wiring - Complete Pin Connection Guide

This section provides exact pin-by-pin wiring instructions. Follow each sub-section in order. All connections are made on the breadboard first for testing before soldering to a permanent PCB.

WARNING: Always disconnect all power before making or changing any wiring connections. The ESP32 operates at 3.3V logic. Do not connect 5V signals directly to GPIO pins unless stated otherwise.

4.1 ESP32 GPIO Pin Assignment Overview

Component	Component Pin	ESP32 Pin	Notes
DS18B20 (Temp)	VCC	3.3V	Power
DS18B20 (Temp)	GND	GND	Ground

DS18B20 (Temp)	DATA	GPIO 4	One-Wire. Add 4.7kΩ pull-up between DATA and VCC
PH4502C Module	VCC	5V (VIN)	Requires 5V
PH4502C Module	GND	GND	Ground
PH4502C Module	Po (Analog Out)	GPIO 34 (ADC)	Analog input only pin
Turbidity Sensor	VCC	5V (VIN)	Requires 5V
Turbidity Sensor	GND	GND	Ground
Turbidity Sensor	Analog Out	GPIO 35 (ADC)	Analog input only pin
Relay Module (CH1)	IN1	GPIO 26	Controls Aerator
Relay Module (CH2)	IN2	GPIO 27	Controls Feeder Motor
Relay Module	VCC	5V (VIN)	Power
Relay Module	GND	GND	Ground
Buzzer	+	GPIO 25	Active buzzer. Use transistor if current > 40mA
Buzzer	-	GND	Ground
LED (Alert)	Anode (+)	GPIO 32	Use 220Ω series resistor
LED (Alert)	Cathode (-)	GND	Ground
Aerator	Live wire	Relay CH1 COM/NO	Relay switches aerator power
Feeder Motor	Power wire	Relay CH2 COM/NO	Relay switches motor power

IMPORTANT: GPIO 34 and GPIO 35 on the ESP32 are input-only ADC pins. Never connect output signals to them. The 4.7kΩ resistor between DS18B20 DATA and VCC is mandatory — the sensor will not work reliably without it.

4.2 DS18B20 Temperature Sensor Wiring

The DS18B20 waterproof version has three wires: Red (VCC), Black (GND), Yellow (Data).

1. Connect RED wire to the ESP32 3.3V pin on the breadboard power rail.
2. Connect BLACK wire to the GND rail on the breadboard.
3. Connect YELLOW wire to breadboard column connected to GPIO 4 on ESP32.
4. **CRITICAL:** Place a 4.7k ohm resistor between the YELLOW data wire and the 3.3V rail. This is the required pull-up resistor for the OneWire protocol. Without it, the sensor will not communicate.

5. Double-check: 3.3V rail -> 4.7k resistor -> GPIO 4 junction; Yellow wire also connects to this same junction.

TIP: The DS18B20 has a unique 64-bit ROM code. If you ever connect multiple temperature sensors, each can be addressed individually on the same data wire. For this project, one sensor is used.

4.3 pH Sensor (PH4502C) Wiring

The PH4502C module has a BNC connector for the glass electrode and screw terminal / pin header outputs.

6. Connect module VCC pin to 5V rail (ESP32 VIN pin or separate 5V source on breadboard).
7. Connect module GND pin to GND rail.
8. Connect module PO (Analog Output) pin to GPIO 34 on ESP32.
9. Plug the pH glass electrode probe into the BNC connector on the module board.
10. The module also has a To (temperature compensation output) and a Do (digital threshold output). Leave these unconnected for basic use.

IMPORTANT: GPIO 34, 35, 36, and 39 on the ESP32 are INPUT ONLY. They cannot be configured as outputs. This is why they are ideal for analog sensor inputs. Do NOT connect any output devices to these pins.

pH Sensor Voltage Divider (REQUIRED):

The PH4502C outputs 0-5V analog signal, but the ESP32 ADC only accepts 0-3.3V (max). You MUST use a voltage divider to protect the ESP32:

11. Use two resistors: R1 = 10k ohm (from pH module PO to GPIO 34) and R2 = 22k ohm (from GPIO 34 to GND).
12. This creates a voltage divider: $V_{out} = V_{in} * (22k / (10k + 22k)) = V_{in} * 0.6875$
13. Maximum input to GPIO 34 becomes: $5V * 0.6875 = 3.44V$ (safe for ESP32).
14. Account for this scaling factor in the pH calculation code (see firmware section).

4.4 Turbidity Sensor Wiring

The turbidity sensor module provides an analog output proportional to turbidity level.

15. Connect sensor module VCC to 5V rail.
16. Connect sensor module GND to GND rail.
17. Connect sensor module SIG or AO (Analog Output) to GPIO 35 on ESP32.
18. Apply the same voltage divider as described for the pH sensor if the module output can exceed 3.3V (measure with a multimeter in clean water to verify max output voltage).
19. Submerge only the sensor probe end in water. Keep all electronics and the PCB dry.

4.5 Relay Module Wiring

The 2-channel relay module controls the aerator (Channel 1) and feeder motor (Channel 2).

Control Side (Low voltage - to ESP32):

20. Connect relay module VCC to 5V rail (VIN pin of ESP32).
21. Connect relay module GND to GND rail.
22. Connect relay module IN1 to GPIO 26 on ESP32.
23. Connect relay module IN2 to GPIO 27 on ESP32.

Load Side (High voltage - to aerator / motor):

Channel 1 (Aerator - if AC powered):

24. Connect the LIVE wire of the aerator's power cord to the COM (Common) terminal of Relay Channel 1.
25. Connect a wire from the NO (Normally Open) terminal of Relay Channel 1 to the wall socket live rail.
26. The NEUTRAL wire of the aerator connects directly to the wall socket neutral (not through relay).

Channel 2 (Feeder Motor - DC powered):

27. Connect the positive (+) wire of the DC motor to the COM terminal of Relay Channel 2.
28. Connect a wire from NO terminal of Relay Channel 2 to the positive of the motor power supply (5-12V DC).
29. Connect the motor negative (-) wire directly to the negative of the motor power supply.

WARNING: If the aerator runs on 220V AC mains power, have a qualified person handle the AC wiring. Use properly insulated wires and ensure the relay is rated for the aerator's current draw. Use a fuse on the AC circuit. All AC wiring must be enclosed in a safe junction box.

RELAY LOGIC NOTE: Most common 2-channel relay modules are ACTIVE LOW, meaning the relay turns ON when the GPIO pin is set LOW (0V) and turns OFF when set HIGH (3.3V). Verify your specific module and adjust the firmware logic accordingly. The code in Section 4 is written for active-low relays.

4.6 Buzzer Wiring

30. Connect the positive leg (+, longer lead) of the active buzzer to GPIO 25 on ESP32.
31. Connect the negative leg (-, shorter lead) to GND rail.
32. An active buzzer contains its own oscillator - it makes sound when DC power is applied. No PWM needed.

4.7 LED Wiring

33. Connect a 330-ohm resistor to GPIO 32 on ESP32 (place on breadboard inline).
34. Connect the other end of the resistor to the ANODE (+, longer leg) of the LED.
35. Connect the CATHODE (-, shorter leg) of the LED to GND rail.
36. Formula: Resistor value = $(3.3V - 2V \text{ forward voltage}) / 0.02A = 65 \text{ ohm minimum}$. 330 ohm provides safe, visible brightness.

4.8 Power Rail Summary

On your breadboard, establish two power rails:

- 3.3V Rail: Connected to ESP32 3.3V pin. Supplies: DS18B20 sensor.
- 5V Rail: Connected to ESP32 VIN pin (when USB-powered). Supplies: pH module, Turbidity module, Relay module.
- GND Rail: Connected to ESP32 GND pin. Common ground for ALL components.

TESTING TIP: Before connecting all sensors at once, connect and test each sensor individually. Use the Serial Monitor in Arduino IDE to verify each sensor gives expected readings before proceeding to the next.

5. ESP32 Firmware Implementation

5.1 Required Libraries

Install the following libraries in Arduino IDE (Sketch > Manage Libraries):

- PubSubClient (by Nick O'Leary) — MQTT client for ESP32
- OneWire + DallasTemperature — DS18B20 temperature sensor
- ArduinoJson — JSON serialization for MQTT payloads
- WiFi (built-in ESP32 core) — Wi-Fi connectivity
- NTPClient + WiFiUdp — Real-time clock via NTP for scheduling
- Preferences (built-in ESP32 core) — Non-volatile storage for config persistence

5.2 Firmware File Structure

```
firmware/
├─ firmware.ino           // Main entry point
├─ config.h               // WiFi SSID, MQTT broker IP, device ID
├─ mqtt_handler.h/.cpp    // MQTT connect, publish, subscribe, callback
├─ sensors.h/.cpp         // Read DS18B20, pH, turbidity
├─ actuators.h/.cpp       // Relay, buzzer, LED control
├─ scheduler.h/.cpp       // NTP time, feeding/aerator schedule logic
└─ config_store.h/.cpp    // Read/write config to NVS (Preferences)
```


5.3 config.h — Configuration File

```
#define WIFI_SSID      "YourWiFiName"
#define WIFI_PASSWORD  "YourWiFiPassword"
#define MQTT_BROKER_IP "192.168.1.100" // IP of your Node.js server
#define MQTT_PORT      1883
#define DEVICE_ID      "pond_01"
#define MQTT_USER      "esp32user"
#define MQTT_PASS      "esp32pass"
#define SENSOR_INTERVAL 10000 // ms between sensor reads

// GPIO Pins
#define PIN_DS18B20  4
#define PIN_PH        34
#define PIN_TURBIDITY 35
#define PIN_RELAY_AERATOR 26
#define PIN_RELAY_FEEDER 27
#define PIN_BUZZER    25
#define PIN_LED        32
```

5.4 Main firmware.ino Loop

```
#include "config.h"
#include "mqtt_handler.h"
#include "sensors.h"
#include "actuators.h"
#include "scheduler.h"
#include "config_store.h"

unsigned long lastSensorRead = 0;

void setup() {
  Serial.begin(115200);
  initActuators(); // Setup relay, buzzer, LED pins
  loadConfigFromNVS(); // Load thresholds & schedules from flash
  connectWiFi(); // Connect to Wi-Fi
  syncNTP(); // Sync time from NTP server
  connectMQTT(); // Connect to Mosquitto broker
```

```
    subscribeTopics();    // Subscribe to commands & config topics
}

void loop() {
    mqttClient.loop();    // Keep MQTT connection alive
    checkSchedules();    // Check if aerator/feeder schedule triggers
    if (millis() - lastSensorRead > SENSOR_INTERVAL) {
        readAndPublishSensors();
        lastSensorRead = millis();
    }
}
```

5.5 MQTT Message Format

5.5.1 Sensor Publish Payload (ESP32 → Server)

Published to topic: fishpond/{device_id}/sensors

```
{
  "device_id": "pond_01",
  "timestamp": 1718000000,
  "temperature": 28.5,
  "ph": 7.2,
  "turbidity": 120,
  "aerator_on": false,
  "feeder_running": false
}
```

5.5.2 Command Payload (Server → ESP32)

Received on topic: fishpond/{device_id}/commands

```
{
  "action": "run_aerator",    // or "run_feeder", "stop_aerator", "stop_feeder"
  "duration_seconds": 30
}
```

5.5.3 Config Update Payload (Server → ESP32)

Received on topic: fishpond/{device_id}/config

```
{
```

```
"temp_max": 32,
"temp_min": 20,
"ph_max": 9.0,
"ph_min": 6.0,
"turbidity_max": 200,
"aerator_auto_duration": 900,
"feeder_schedules": [
  { "hour": 7, "minute": 0, "duration_s": 10 },
  { "hour": 18, "minute": 0, "duration_s": 10 }
],
"aerator_schedules": [
  { "hour": 6, "minute": 0, "duration_s": 300 }
]
}
```

5.6 Threshold Alert Logic (on ESP32)

```
void checkThresholds(float temp, float ph, float turbidity) {
  bool alert = false;
  String alertMsg = "";

  if (temp > config.temp_max || temp < config.temp_min) {
    alert = true; alertMsg += "TEMP:" + String(temp) + " ";
    if (temp > config.temp_max) runAerator(config.aerator_auto_duration);
  }
  if (ph > config.ph_max || ph < config.ph_min) {
    alert = true; alertMsg += "PH:" + String(ph) + " ";
  }
  if (turbidity > config.turbidity_max) {
    alert = true; alertMsg += "TURB:" + String(turbidity);
  }
  if (alert) {
    triggerLocalAlert(); // Buzzer + LED
    publishAlert(alertMsg); // Publish to fishpond/{id}/alerts
  }
}
```

5.7 NVS Config Persistence

Use the ESP32 Preferences library to save and restore configuration across reboots. This ensures schedules and thresholds survive power cuts.

```
#include <Preferences.h>

Preferences prefs;

void saveConfigToNVS() {
  prefs.begin("fishpond", false);
  prefs.putFloat("temp_max", config.temp_max);
  prefs.putFloat("ph_min", config.ph_min);
  // ... store all config values
  prefs.end();
}

void loadConfigFromNVS() {
  prefs.begin("fishpond", true);
  config.temp_max = prefs.getFloat("temp_max", 32.0); // default 32
  config.ph_min   = prefs.getFloat("ph_min", 6.0);
  // ... load all config values
  prefs.end();
}
```

6. Backend — Node.js + Express.js Server

6.1 Project Structure

```
backend/
├── server.js           // Entry point: Express + Socket.IO + MQTT
├── .env               // Environment variables
├── config/
│   └── db.js          // MongoDB connection
├── models/
│   ├── SensorReading.js // MongoDB schema for sensor data
│   ├── Device.js       // Device config schema
│   └── User.js         // User schema
├── routes/
│   ├── auth.js        // POST /api/auth/register, /login
│   ├── sensors.js     // GET /api/sensors/history
│   └── device.js       // GET/PUT /api/device/config
```

```
|   └─ control.js          // POST /api/control/aerator, /feeder
|   └─ alerts.js          // GET /api/alerts
└─ middleware/
    └─ auth.js            // JWT verification middleware
└─ mqtt/
    └─ mqttClient.js      // Mosquitto subscribe/publish logic
└─ push/
    └─ webpush.js         // Web Push notification sender
```

6.2 Environment Variables (.env)

```
PORT=3000
MONGO_URI=mongodb://localhost:27017/fishpond
MQTT_BROKER_URL=mqtt://localhost:1883
MQTT_USERNAME=esp32user
MQTT_PASSWORD=esp32pass
JWT_SECRET=your_very_long_jwt_secret_here
VAPID_PUBLIC_KEY=your_vapid_public_key
VAPID_PRIVATE_KEY=your_vapid_private_key
VAPID_EMAIL=mailto:you@example.com
```

6.3 server.js — Main Entry Point

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const cors = require('cors');
const connectDB = require('./config/db');
const mqttClient = require('./mqtt/mqttClient');
require('dotenv').config();

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: '*' } });

app.use(cors());
app.use(express.json());
```

```
// Routes
app.use('/api/auth',    require('./routes/auth'));
app.use('/api/sensors', require('./routes/sensors'));
app.use('/api/device',  require('./routes/device'));
app.use('/api/control', require('./routes/control'));
app.use('/api/alerts',  require('./routes/alerts'));

// Start
connectDB();
mqttClient.init(io); // pass Socket.IO instance for real-time relay
server.listen(process.env.PORT, () => {
  console.log(`Server running on port ${process.env.PORT}`);
});
```

6.4 MQTT Client (mqtt/mqttClient.js)

```
const mqtt      = require('mqtt');
const SensorReading = require('../models/SensorReading');
const { sendPushNotification } = require('../push/webpush');

let mqttClient;

function init(io) {
  mqttClient = mqtt.connect(process.env.MQTT_BROKER_URL, {
    username: process.env.MQTT_USERNAME,
    password: process.env.MQTT_PASSWORD,
  });

  mqttClient.on('connect', () => {
    console.log('Connected to Mosquitto broker');
    mqttClient.subscribe('fishpond/+/sensors');
    mqttClient.subscribe('fishpond/+/alerts');
    mqttClient.subscribe('fishpond/+/status');
  });

  mqttClient.on('message', async (topic, payload) => {
    const data = JSON.parse(payload.toString());
```

```
    if (topic.endsWith('/sensors')) {
      // Save to MongoDB
      await SensorReading.create(data);
      // Broadcast to all PWA clients via Socket.IO
      io.emit('sensor_update', data);
    }

    if (topic.endsWith('/alerts')) {
      // Send Web Push Notification to subscribed PWA clients
      await sendPushNotification(data.device_id, data.message);
      io.emit('alert', data);
    }
  });
}

function publishCommand(deviceId, payload) {
  mqttClient.publish(`fishpond/${deviceId}/commands`,
    JSON.stringify(payload), { qos: 1 });
}

module.exports = { init, publishCommand };
```

6.5 MongoDB Schemas

6.5.1 SensorReading Schema

```
const mongoose = require('mongoose');
const SensorReadingSchema = new mongoose.Schema({
  device_id: { type: String, required: true, index: true },
  timestamp: { type: Date, default: Date.now, index: true },
  temperature: { type: Number },
  ph: { type: Number },
  turbidity: { type: Number },
  aerator_on: { type: Boolean },
  feeder_running: { type: Boolean },
}, { expireAfterSeconds: 2592000 }); // 30-day TTL auto-delete

module.exports = mongoose.model('SensorReading', SensorReadingSchema);
```

6.5.2 Device Config Schema

```
const DeviceSchema = new mongoose.Schema({
  device_id: { type: String, required: true, unique: true },
  owner_user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  temp_max: { type: Number, default: 32 },
  temp_min: { type: Number, default: 20 },
  ph_max: { type: Number, default: 9.0 },
  ph_min: { type: Number, default: 6.0 },
  turbidity_max: { type: Number, default: 200 },
  aerator_auto_duration: { type: Number, default: 900 },
  feeder_schedules: [{ hour: Number, minute: Number, duration_s: Number }],
  aerator_schedules: [{ hour: Number, minute: Number, duration_s: Number }],
  push_subscription: { type: Object }, // Web Push subscription object
});
```

6.6 REST API Reference

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register a new user account
POST	/api/auth/login	None	Login, returns JWT token
GET	/api/sensors/history	JWT	Get sensor history (query: device_id, range)
GET	/api/sensors/latest	JWT	Get latest sensor reading for a device
GET	/api/device/config	JWT	Get device configuration
PUT	/api/device/config	JWT	Update thresholds and schedules
POST	/api/control/aerator	JWT	Run/stop aerator manually (duration in body)
POST	/api/control/feeder	JWT	Run/stop feeder manually (duration in body)
POST	/api/device/push-subscribe	JWT	Save Web Push subscription
GET	/api/alerts	JWT	Get alert history log

6.7 Control Route — Manual Actuator Command

```
// routes/control.js
const express = require('express');
```



```
const router      = express.Router();
const authMiddleware = require('../middleware/auth');
const { publishCommand } = require('../mqtt/mqttClient');

router.post('/aerator', authMiddleware, async (req, res) => {
  const { device_id, action, duration_seconds } = req.body;
  // action: 'run_aerator' or 'stop_aerator'
  publishCommand(device_id, { action, duration_seconds });
  res.json({ success: true, message: 'Command sent via MQTT' });
});

router.post('/feeder', authMiddleware, async (req, res) => {
  const { device_id, action, duration_seconds } = req.body;
  publishCommand(device_id, { action, duration_seconds });
  res.json({ success: true, message: 'Command sent via MQTT' });
});

module.exports = router;
```

6.8 Web Push Notification Setup

6.8.1 Generate VAPID Keys

```
npm install web-push
npx web-push generate-vapid-keys
# Copy the output PUBLIC and PRIVATE keys into your .env file
```

6.8.2 webpush.js

```
const webpush = require('web-push');
const Device = require('../models/Device');

webpush.setVapidDetails(
  process.env.VAPID_EMAIL,
  process.env.VAPID_PUBLIC_KEY,
  process.env.VAPID_PRIVATE_KEY
);

async function sendPushNotification(deviceId, message) {
  const device = await Device.findOne({ device_id: deviceId });
```

```
if (!device || !device.push_subscription) return;
const payload = JSON.stringify({ title: 'Fish Pond Alert', body: message });
await webpush.sendNotification(device.push_subscription, payload);
}

module.exports = { sendPushNotification };
```

6.9 Install & Run Backend

```
cd backend
npm init -y
npm install express mongoose mqtt socket.io cors dotenv web-push jsonwebtoken bcryptjs

# Start Mosquitto MQTT Broker (install separately, see Section 5.1)
mosquitto -c /etc/mosquitto/mosquitto.conf -d

# Start MongoDB (if self-hosted)
mongod --dbpath /var/lib/mongodb

# Start Node.js backend
node server.js

# Or use PM2 for production (auto-restart, logging)
npm install -g pm2
pm2 start server.js --name fishpond-backend
pm2 save && pm2 startup
```

7. Mosquitto MQTT Broker Setup

7.1 Installation

```
# Ubuntu / Debian (server)
sudo apt update
sudo apt install mosquitto mosquitto-clients -y
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
```

```
# Verify it is running
sudo systemctl status mosquitto
```

7.2 Configuration (/etc/mosquitto/mosquitto.conf)

```
listener 1883
allow_anonymous false
password_file /etc/mosquitto/passwd

# Optional: Enable WebSocket (port 9001) for browser MQTT clients
listener 9001
protocol websockets
```

7.3 Create MQTT User

```
sudo mosquitto_passwd -c /etc/mosquitto/passwd esp32user
# Enter password when prompted (use the same password in config.h and .env)
sudo systemctl restart mosquitto
```

For production: Deploy the backend and Mosquitto broker on the same VPS (e.g., DigitalOcean, Linode, or a local Raspberry Pi on the same network). The ESP32 MQTT_BROKER_IP must point to the server's LAN IP (or public IP if deployed remotely). Open port 1883 in the server firewall for MQTT.

8. Frontend — React Progressive Web App (PWA)

8.1 Project Setup

```
npx create-react-app fishpond-pwa
cd fishpond-pwa
npm install axios socket.io-client recharts react-router-dom
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

8.2 Folder Structure

```
src/
├─ index.js           // React entry point
├─ App.js             // Router setup
├─ api/
│   ├─ axiosConfig.js // Axios base URL + JWT interceptor
│   └─ api.js         // All API call functions
├─ context/
│   └─ AuthContext.js // JWT auth context + login/logout
├─ hooks/
│   └─ useSocket.js   // Socket.IO connection hook
├─ pages/
│   ├─ Login.jsx
│   ├─ Dashboard.jsx // Main real-time sensor dashboard
│   ├─ AeratorControl.jsx
│   ├─ FeederControl.jsx
│   ├─ AlertSettings.jsx
│   ├─ History.jsx
│   └─ DeviceSettings.jsx
├─ components/
│   ├─ SensorCard.jsx // Color-coded temp/pH/turbidity card
│   ├─ ScheduleList.jsx // Add/remove time schedules
│   └─ Navbar.jsx
└─ serviceWorker/
    └─ sw.js          // Service worker for PWA + push
```

8.3 Socket.IO Real-Time Hook (hooks/useSocket.js)

```
import { useEffect, useState } from 'react';
import { io } from 'socket.io-client';

const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;

export function useSocket() {
  const [sensorData, setSensorData] = useState(null);
  const [alert, setAlert] = useState(null);

  useEffect(() => {
    const socket = io(BACKEND_URL);
    socket.on('sensor_update', (data) => setSensorData(data));
```

```
    socket.on('alert', (data) => setAlert(data));  
    return () => socket.disconnect();  
  }, []);  
  
  return { sensorData, alert };  
}
```

8.4 Dashboard Page (pages/Dashboard.jsx)

```
import { useSocket } from '../hooks/useSocket';  
import SensorCard from '../components/SensorCard';  
import { postControl } from '../api/api';  
  
export default function Dashboard() {  
  const { sensorData, alert } = useSocket();  
  
  const runAerator = async () => {  
    await postControl('/api/control/aerator', {  
      device_id: 'pond_01', action: 'run_aerator', duration_seconds: 30  
    });  
  };  
  
  return (  
    <div className='p-4 grid grid-cols-1 md:grid-cols-3 gap-4'>  
      {sensorData && <>  
        <SensorCard label='Temperature'  
          value={sensorData.temperature} unit='°C'  
          min={20} max={32} />  
        <SensorCard label='pH Level'  
          value={sensorData.ph} unit='pH'  
          min={6.0} max={9.0} />  
        <SensorCard label='Turbidity'  
          value={sensorData.turbidity} unit='NTU'  
          min={0} max={200} />  
      </>  
      {alert && <div className='col-span-3 bg-red-100 border p-3 rounded'>  
        ALERT: {alert.message}  
      </div>  
      <button onClick={runAerator}>
```

```
        className='bg-blue-600 text-white rounded p-2'>
        Run Aerator Now
    </button>
</div>
);
}
```

8.5 SensorCard Component

```
export default function SensorCard({ label, value, unit, min, max }) {
    const status = value < min || value > max ? 'red'
        : (value < min*1.05 || value > max*0.95) ? 'yellow' : 'green';
    const bg = { red: 'bg-red-100 border-red-500',
        yellow: 'bg-yellow-100 border-yellow-500',
        green: 'bg-green-100 border-green-500' }[status];
    return (
        <div className={`border-2 rounded-xl p-4 ${bg}`}>
            <p className='text-sm font-semibold text-gray-600'>{label}</p>
            <p className='text-4xl font-bold mt-1'>{value?.toFixed(1)} {unit}</p>
            <p className='text-xs text-gray-500 mt-1'>Normal: {min} - {max} {unit}</p>
        </div>
    );
}
```

8.6 Making the App a PWA

Step 1: Create manifest.json in /public/

```
{
  "short_name": "FishPond",
  "name": "IoT Fish Pond Manager",
  "icons": [{ "src": "icon-192.png", "sizes": "192x192", "type": "image/png" }],
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#1A3A6B"
}
```

Step 2: Register Service Worker in index.js

```
if ('serviceWorker' in navigator) {
```

```
navigator.serviceWorker.register('/sw.js').then(reg => {
  console.log('Service Worker registered:', reg.scope);
});
}
```

Step 3: Service Worker (public/sw.js) for Push

```
self.addEventListener('push', event => {
  const data = event.data.json();
  self.registration.showNotification(data.title, {
    body: data.body,
    icon: '/icon-192.png'
  });
});
```

Step 4: Subscribe User to Push (in DeviceSettings.jsx)

```
async function subscribeToPush() {
  const reg = await navigator.serviceWorker.ready;
  const sub = await reg.pushManager.subscribe({
    userVisibleOnly: true,
    applicationServerKey: process.env.REACT_APP_VAPID_PUBLIC_KEY
  });
  // Send sub object to backend to save in MongoDB
  await api.post('/api/device/push-subscribe', {
    device_id: 'pond_01', subscription: sub
  });
}
```

8.7 Deployment

```
# Build the React app
npm run build

# Option A: Serve with Node.js (add to server.js)
app.use(express.static('fishpond-pwa/build'));
app.get('*', (req, res) => res.sendFile('build/index.html', { root: './fishpond-pwa' }));

# Option B: Deploy PWA on Vercel (free, HTTPS included)
npm install -g vercel
cd fishpond-pwa && vercel
```

```
# HTTPS is REQUIRED for PWA install prompt and Web Push – always use HTTPS in production.
```

HTTPS is mandatory for PWA install prompts and Web Push Notifications to work on mobile devices. For production, deploy the Node.js backend behind an Nginx reverse proxy with Let's Encrypt SSL (free certificate). For local testing, Chrome allows PWA on localhost without HTTPS.

9. Authentication — JWT + Firebase Auth

9.1 JWT-Based Auth Flow

The primary authentication mechanism is JWT. Every protected API route requires a valid Bearer token in the Authorization header. Firebase Auth can be used optionally as an identity provider (Google sign-in), with the Firebase ID token exchanged for a local JWT.

9.2 User Registration & Login (routes/auth.js)

```
const bcrypt = require('bcryptjs');
const jwt    = require('jsonwebtoken');
const User   = require('../models/User');

// POST /api/auth/register
router.post('/register', async (req, res) => {
  const { email, password } = req.body;
  const hashed = await bcrypt.hash(password, 10);
  const user = await User.create({ email, password: hashed });
  res.json({ success: true });
});

// POST /api/auth/login
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !await bcrypt.compare(password, user.password))
    return res.status(401).json({ error: 'Invalid credentials' });
  const token = jwt.sign({ userId: user._id }, process.env.JWT_SECRET, { expiresIn: '30d' });
});
```



```
    res.json({ token });  
  });  
};
```

9.3 JWT Middleware (middleware/auth.js)

```
const jwt = require('jsonwebtoken');  
  
module.exports = (req, res, next) => {  
  const header = req.headers['authorization'];  
  if (!header) return res.status(401).json({ error: 'No token' });  
  const token = header.split(' ')[1];  
  try {  
    req.user = jwt.verify(token, process.env.JWT_SECRET);  
    next();  
  } catch {  
    res.status(401).json({ error: 'Invalid token' });  
  }  
};
```

9.4 Storing JWT in PWA

```
// AuthContext.js - store token in memory (safest for PWA)  
// Do not use localStorage for sensitive tokens in production.  
const [token, setToken] = useState(sessionStorage.getItem('jwt') || null);  
  
const login = async (email, password) => {  
  const res = await axios.post('/api/auth/login', { email, password });  
  sessionStorage.setItem('jwt', res.data.token);  
  setToken(res.data.token);  
};  
  
// axiosConfig.js - auto-attach token to every request  
axios.interceptors.request.use(config => {  
  const token = sessionStorage.getItem('jwt');  
  if (token) config.headers.Authorization = `Bearer ${token}`;  
  return config;  
});
```

10. pH Sensor Calibration

The PH4502C outputs an analog voltage proportional to pH. The ESP32 ADC reads this as a raw 12-bit value (0-4095). A two-point linear calibration must be performed using standard buffer solutions.

10.1 Calibration Procedure

1. Prepare two standard buffer solutions: pH 4.0 and pH 7.0.
2. Rinse the pH probe with distilled water and immerse it in the pH 7.0 buffer solution.
3. Wait 2 minutes for the reading to stabilize. Record the raw ADC value from the ESP32 Serial Monitor (call `Serial.println(analogRead(PIN_PH))`).
4. Rinse with distilled water and immerse in pH 4.0 buffer solution.
5. Wait 2 minutes. Record the raw ADC value.
6. Use the two data points to calculate a linear mapping: $pH = (rawValue - raw_at_7) * ((4.0 - 7.0) / (raw_at_4 - raw_at_7)) + 7.0$
7. Store the two calibration constants (`raw_at_4`, `raw_at_7`) in NVS using the Preferences library.

10.2 Calibration Code

```
float readPH() {  
    int raw = analogRead(PIN_PH);  
    // Load calibration constants from NVS  
    float raw_at_7 = prefs.getFloat('ph_raw7', 2048.0);  
    float raw_at_4 = prefs.getFloat('ph_raw4', 1500.0);  
    float ph = (raw - raw_at_7) * ((4.0 - 7.0) / (raw_at_4 - raw_at_7)) + 7.0;  
    return constrain(ph, 0.0, 14.0);  
}
```

11. Full System Setup Checklist

11.1 Hardware Assembly

- ☐ Wire DS18B20 to GPIO 4 with 4.7kΩ pull-up resistor between DATA and 3.3V
- ☐ Wire PH4502C to GPIO 34 (5V power from VIN)
- ☐ Wire Turbidity Sensor to GPIO 35 (5V power from VIN)
- ☐ Wire Relay IN1 to GPIO 26 (Aerator), IN2 to GPIO 27 (Feeder)
- ☐ Wire Relay VCC to 5V (VIN), GND to GND
- ☐ Wire Buzzer to GPIO 25 (+), GND (-)
- ☐ Wire LED to GPIO 32 via 220Ω resistor
- ☐ Connect Aerator to Relay CH1 COM/NO terminals

- ☐ Connect Feeder Motor to Relay CH2 COM/NO terminals
- ☐ House all electronics in waterproof IP65 enclosure

11.2 Server Setup

- ☐ Install Node.js v18+ on server
- ☐ Install MongoDB (or create free MongoDB Atlas cluster)
- ☐ Install and configure Mosquitto MQTT Broker
- ☐ Create MQTT user credentials (match config.h and .env)
- ☐ Clone backend code and run npm install
- ☐ Fill in .env file with all required variables
- ☐ Generate VAPID keys and add to .env
- ☐ Start server with PM2 (pm2 start server.js)
- ☐ Configure firewall: open port 3000 (API/Socket.IO) and 1883 (MQTT)
- ☐ Set up HTTPS with Nginx + Let's Encrypt (mandatory for PWA push)

11.3 ESP32 Firmware

- ☐ Install Arduino IDE and ESP32 board support
- ☐ Install all required libraries (Section 3.1)
- ☐ Fill in config.h with WiFi credentials and MQTT broker IP
- ☐ Upload firmware to ESP32
- ☐ Open Serial Monitor at 115200 baud to verify Wi-Fi and MQTT connection
- ☐ Perform pH sensor two-point calibration (Section 8)
- ☐ Verify sensor readings appear in Serial Monitor

11.4 PWA Frontend

- ☐ Run npm install in fishpond-pwa/
- ☐ Create .env with REACT_APP_BACKEND_URL and REACT_APP_VAPID_PUBLIC_KEY
- ☐ Run npm run build
- ☐ Deploy build/ to server or Vercel
- ☐ Open PWA in Chrome on Android and tap 'Add to Home Screen' to install
- ☐ Register account, login, and verify dashboard shows live sensor data
- ☐ Go to Device Settings and subscribe to Web Push Notifications
- ☐ Test manual aerator and feeder control from dashboard
- ☐ Set feeding and aerator schedules
- ☐ Trigger a threshold breach and verify push notification is received

12. Testing & Troubleshooting

12.1 Testing MQTT Connection

```
# On server - subscribe to all fishpond messages
mosquitto_sub -h localhost -u esp32user -P esp32pass -t 'fishpond/#' -v

# On another terminal - publish a test message manually
mosquitto_pub -h localhost -u esp32user -P esp32pass \
  -t 'fishpond/pond_01/sensors' \
  -m '{"device_id":"pond_01","temperature":28.5,"ph":7.2,"turbidity":100}'
```

12.2 Testing REST API

```
# Login and get JWT token
curl -X POST http://localhost:3000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"user@example.com","password":"pass123"}'

# Get latest sensor reading (replace TOKEN)
curl http://localhost:3000/api/sensors/latest?device_id=pond_01 \
  -H 'Authorization: Bearer TOKEN'
```

12.3 Common Issues & Solutions

Problem	Likely Cause	Solution
ESP32 not connecting to MQTT	Wrong broker IP or credentials in config.h	Check MQTT_BROKER_IP and run mosquitto_sub on server to confirm broker is listening
pH readings wildly inaccurate	Probe not calibrated or dirty	Re-calibrate with fresh buffer solutions; clean probe with distilled water
PWA not showing live data	Socket.IO not connected	Check REACT_APP_BACKEND_URL; open browser DevTools > Network for WS errors
Push notifications not received	No HTTPS / VAPID keys wrong / user not subscribed	Ensure HTTPS is active; verify VAPID keys match between .env and PWA .env; re-subscribe in DeviceSettings
Relay not activating	GPIO logic inverted (active LOW relay)	Most relay modules are active LOW; set GPIO LOW to activate, HIGH to deactivate

Feeder not running on schedule	ESP32 NTP not synced	Verify internet access; check NTP sync in Serial Monitor at boot
MongoDB connection refused	MongoDB not running or URI wrong	Run mongod --dbpath; verify MONGO_URI in .env

13. Production Deployment (Recommended Setup)

13.1 Server Recommendation

For a stable production setup, deploy on a Linux VPS (Ubuntu 22.04 LTS) — DigitalOcean Droplet or a dedicated Raspberry Pi 4 on the local network. Minimum specs: 1 vCPU, 1GB RAM, 20GB SSD. MongoDB Atlas free tier eliminates local DB management.

13.2 Nginx Reverse Proxy + HTTPS

```
# Install Nginx
sudo apt install nginx certbot python3-certbot-nginx -y

# /etc/nginx/sites-available/fishpond
server {
    server_name fishpond.yourdomain.com;
    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade'; # Required for Socket.IO
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}

# Enable and get SSL certificate (free)
sudo certbot --nginx -d fishpond.yourdomain.com
sudo nginx -t && sudo systemctl reload nginx
```

13.3 PM2 Process Management

```
pm2 start server.js --name fishpond-backend
pm2 save           # Persist across reboots
pm2 startup        # Generate startup script
pm2 logs fishpond-backend # View logs
pm2 monit          # Real-time monitoring dashboard
```

End of Engineering Implementation Guide

IoT Fish Pond Monitoring & Management System | Engineering Guide v2.0
